

LLM Foundation — Interview & Revision Notes

1. LLM Fundamentals

LLM (Large Language Model) হলো এমন একটি AI model যা মানুষের ভাষার text বুঝতে, বিশ্লেষণ করতে এবং নতুন text generate করতে পারে।

LLM সাধারণত input নেয় এবং তার ভিত্তিতে output generate করে।

User Input



LLM



Generated Output

Backend developer হিসেবে আমাদের সাধারণত LLM train করতে হয় না। API ব্যবহার করে existing LLM application-এর সাথে integrate করি।

Interview:

LLM is a Large Language Model that can understand and generate human-like text. In backend applications, we usually interact with LLMs through APIs rather than training them from scratch.

2. LLM API Fundamentals

LLM API হলো application এবং LLM-এর মধ্যে communication interface।

আমাদের backend LLM API-তে request পাঠায় এবং LLM response পাঠায়।

Backend



HTTP Request



LLM API



LLM



HTTP Response



Backend

API-এর মাধ্যমে আমরা model, input, instructions ইত্যাদি পাঠাতে পারি।

Interview:

An LLM API allows an application to communicate with a language model through HTTP requests and responses.

3. OpenAI API

OpenAI API ব্যবহার করে backend application থেকে OpenAI-এর models ব্যবহার করা যায়।

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
response = client.responses.create(  
    model="gpt-5",  
    input="Explain Python."  
)
```

```
print(response.output_text)
```

এখানে:

- client → API client
- model → কোন model ব্যবহার হবে
- input → LLM-এর কাছে পাঠানো request
- output_text → generated response

Interview:

I can integrate OpenAI models into a Python backend using the OpenAI SDK and send requests through the Responses API.

4. Gemini API

Gemini API হলো Google-এর Gemini models-এর সাথে application integrate করার উপায়।

```
from google import genai
```

```
client = genai.Client()
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents="Explain FastAPI."  
)
```

```
print(response.text)
```

OpenAI এবং Gemini আলাদা provider হলেও basic concept একই:

Application



Provider API



LLM



Response

5. API Key & Environment Variables

API key হলো application-এর authentication credential।

এটা source code-এ রাখা উচিত নয়।



```
client = OpenAI(api_key="secret-key")
```



```
OPENAI_API_KEY=your-secret-key
```

```
import os
```

```
api_key = os.getenv("OPENAI_API_KEY")
```

.env সাধারণত .gitignore-এ রাখা হয়।

```
.env
```

Interview:

I store API keys in environment variables instead of hardcoding them, so secrets are not exposed in the source code or Git repository.

6. HTTP Request / Response

API communication সাধারণত HTTP request এবং response-এর মাধ্যমে হয়।

Request



Server/API



Response

Example:

POST /chat

Request:

{

```
"message": "Hello"
}
```

Response:

```
{
  "answer": "Hello! How can I help?"
}
```

LLM API integration বুঝতে HTTP request/response জানা গুরুত্বপূর্ণ।

7. JSON

JSON (JavaScript Object Notation) হলো structured data exchange-এর একটি common format।

```
{
  "category": "billing",
  "priority": "high",
  "sentiment": "negative"
}
```

Python:

```
data = {
    "category": "billing",
    "priority": "high"
}
```

```
print(data["category"])
```

LLM backend application-এ JSON খুব গুরুত্বপূর্ণ কারণ AI output structured করে backend-এ ব্যবহার করা যায়।

8. System Message

System message LLM-এর **behavior এবং instructions** define করে।

```
instructions = ""
```

You are a professional customer support assistant.

Be concise and polite.

```
"""
```

এটা LLM-কে বলে:

তুমি কী role পালন করবে এবং কীভাবে response দেবে।

System Instruction

↓

User Message

↓

LLM

↓

Response

Interview:

A system message defines the behavior, role, and high-level instructions that guide the model's responses.

9. User Message

User message হলো user-এর actual request বা input।

System:

You are a Python teacher.

User:

Explain FastAPI.

এখানে:

System Message → কীভাবে behave করবে

User Message → কী করতে হবে

Backend application-এ user input সাধারণত API request থেকে আসে।

10. Prompt Engineering

Prompt engineering হলো LLM-কে clear এবং effective instruction দেওয়ার process।

থারাপ:

Write about Python.

ভালো:

Explain Python to a beginner.

Requirements:

- Use simple English
- Give 3 points
- Give one example
- Keep it under 100 words

ভালো prompt সাধারণত:

Role



Task



Context



Requirements



Expected Output

Interview:

Prompt engineering is the process of designing clear instructions and context to get more accurate, consistent, and useful model outputs.

11. Context

Context হলো LLM-কে দেওয়া relevant information, যা model current response generate করার সময় ব্যবহার করে।

Example:

Context:

The user is a beginner in Python.

Question:

Explain FastAPI.

LLM context অনুযায়ী beginner-friendly answer দিতে পারে।

Conversation-এর ক্ষেত্রে previous messages-ও context হিসেবে কাজ করতে পারে।

User → My name is Almas.

AI → Nice to meet you.

User → What is my name?

AI → Your name is Almas.

Important: LLM automatically সব previous conversation জানে না। Application-কে প্রয়োজনীয় conversation/context model-এর কাছে পাঠাতে হয়।

12. Temperature

Temperature model-এর output-এর **randomness/variation** control করে।

Conceptually:

Lower Temperature



More predictable

More consistent

Higher Temperature



More variation

More creative

উদাহরণ:

Classification

→ Controlled output দরকার

Creative writing

→ More variation দরকার

Important: সব model/API একইভাবে temperature support বা expose নাও করতে পারে। তাই specific model-এর documentation অনুযায়ী parameter ব্যবহার করতে হবে।

13. Tokens

LLM text-কে ছোট ছোট units বা **tokens** হিসেবে process করে।

Input



Tokens



LLM

↓

Tokens

↓

Output

API usage বুঝতে tokens গুরুত্বপূর্ণ।

Input Tokens

+

Output Tokens

=

Total Tokens

Example:

Input = 100 tokens

Output = 50 tokens

Total = 150 tokens

Tokens গুরুত্বপূর্ণ কারণ:

- Cost
- Context limits
- Performance
- Usage monitoring

14. Model Selection

Model selection মানে কাজ অনুযায়ী suitable model নির্বাচন করা।

Consider করতে হবে:

Task

↓

Quality

Speed

Cost

Context requirements

Capabilities



Model

Example:

Simple classification

→ Fast/efficient model

Complex reasoning

→ More capable model

Large context

→ Large-context model

Interview:

I select a model based on the task requirements, latency, cost, context size, and required reasoning or output quality.

15. Chat Completion

Chat completion বলতে conversational interaction-এর জন্য messages/instructions ব্যবহার করে model response generate করাকে বোঝায়।

Concept:

System



User

↓

Assistant

↓

User

↓

Assistant

Example:

User: What is Python?

AI: Python is a programming language.

User: What can I build with it?

AI: You can build web applications...

Backend-এ conversation maintain করতে previous relevant messages/context আবার model-কে দিতে হয়।

16. Structured JSON Output

LLM-কে নির্দিষ্ট structure অনুযায়ী output দিতে বলা হলো structured output।

Normal:

The email is about billing and the customer is unhappy.

Structured:

```
{  
  "category": "billing",  
  "priority": "high",  
  "sentiment": "negative"  
}
```

Backend application-এর জন্য structured output বেশি useful কারণ সহজে process করা যায়।

LLM



Structured JSON



Backend



Database/API

17. Pydantic Output Validation

LLM outputকে blindly trust করা উচিত নয়।

Pydantic দিয়ে expected structure এবং data type validate করা যায়।

```
from pydantic import BaseModel
```

```
class Classification(BaseModel):
```

```
    category: str
```

```
    priority: str
```

```
    sentiment: str
```

তারপর:

```
data = Classification.model_validate({
```

```
    "category": "billing",
```

```
    "priority": "high",
```

```
    "sentiment": "negative"
```

```
})
```

যদি required field missing হয় বা invalid type/value আসে, validation error হতে পারে।

LLM Output

↓

Pydantic Validation

↓

Valid?

|— Yes → Continue

└— No → Handle Error

Interview:

I use Pydantic to validate structured LLM output before passing it to business logic or storing it in the database.

18. Streaming Response

Normal response:

Request

↓

LLM generates everything

↓

Complete response

↓

User

Streaming:

Request

↓

LLM

↓

Chunk 1

↓

Chunk 2



Chunk 3



User

অর্থাৎ complete response-এর জন্য অপেক্ষা না করে generated text-এর অংশগুলো gradually client-এর কাছে পাঠানো হয়।

Useful for:

- Chatbots
- Long responses
- AI assistants

Interview:

Streaming allows the application to send generated output incrementally instead of waiting for the complete response, improving perceived responsiveness.

19. API Error Handling

LLM API call সবসময় সফল হবে না।

Possible errors:

Invalid API Key

Bad Request

Rate Limit

Timeout

Network Error

Server Error

Python:

try:

```
response = client.responses.create(  
    model="gpt-5",  
    input="Hello"  
)
```

except Exception as e:

```
    print("LLM request failed")
```

Production application-এ error অনুযায়ী appropriate handling করতে হবে।

API Request

↓

Success?

└— Yes → Response

└— No → Error Handling

20. Rate Limit Handling

Rate limit হলো নির্দিষ্ট সময়ের মধ্যে API request-এর সীমা।

যদি খুব বেশি request পাঠানো হয়:

Too Many Requests

↓

Rate Limit

↓

Request rejected

সাধারণ approach:

Request

↓

Rate Limit Error

↓

Wait

↓

Retry

Simple example:

```
import time
```

```
for attempt in range(3):
```

```
    try:
```

```
        response = client.responses.create(
```

```
            model="gpt-5",
```

```
            input="Hello"
```

```
        )
```

```
        break
```

```
    except Exception:
```

```
        time.sleep(2)
```

Production-এ exponential backoff ব্যবহার করা বেশি appropriate।

21. Token & Cost Management

LLM API usage-এর cost token usage-এর সাথে সম্পর্কিত।

Input Tokens

+

Output Tokens

↓

Usage



Cost

তাই usage track করা গুরুত্বপূর্ণ।

```
print(response.usage)
```

Cost কমানোর উপায়:

Unnecessary Context বাদ



Short + Clear Prompt



Unnecessary Output কমানো



Suitable Model নির্বাচন



Cost কমানো

Interview:

I monitor input and output token usage and optimize prompts, context, output length, and model selection to control LLM costs.

22. Basic LLM Security

LLM application-এ কয়েকটি basic security principle follow করতে হবে।

API Key Protection

API Key



Environment Variable



Never commit to Git

User Input Validation

```
if len(user_input) > 5000:  
    raise ValueError("Input too long")
```

Sensitive Data Protection

অপ্রয়োজনে LLM-এ পাঠাবে না:

Passwords

API Keys

Payment credentials

Private secrets

Output Validation

LLM

↓

Pydantic

↓

Validation

↓

Business Logic

Prompt Injection Awareness

User-এর input-কে blindly trusted instruction হিসেবে treat করা যাবে না।

User Input

↓

Validation / Controls

↓

Prompt

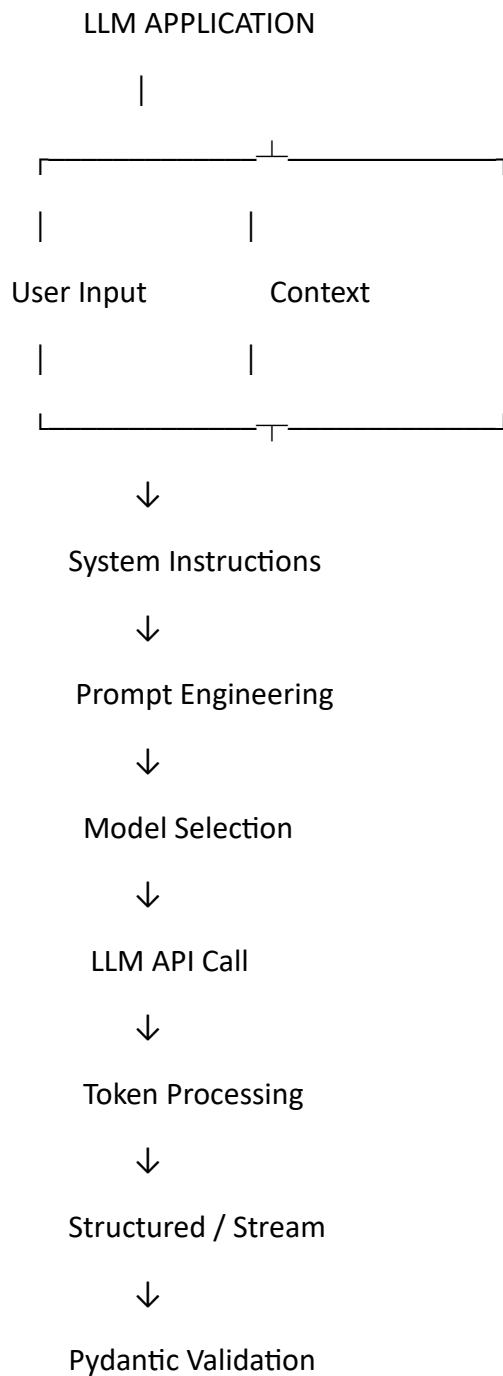
↓

LLM

Interview:

For LLM security, I protect API keys, validate user input, avoid exposing sensitive data, validate model outputs, and design the application to reduce risks such as prompt injection.

পুরো LLM Foundation একসাথে





Error / Rate Handling



Cost Monitoring



API Response

Interview-এ পুরোটা একসাথে বললে

I integrate LLMs into backend applications through APIs. I manage system instructions, user prompts, context, model selection, structured outputs and streaming. I validate AI-generated data with Pydantic, handle API errors and rate limits, monitor token usage and cost, and protect API credentials and user data through basic security practices.